

Unit 2 Practice Test

Practice 2.1

Mike's teacher told his class that a flipped coin lands on "heads" half the time and "tails" the other half. "That means that if I flip a coin 100 times, it should land on heads exactly fifty times!" said Mike. Mike's teacher knows that the law of probability does not quite work that way. To demonstrate this principle, she decides to write a program.

Program

Write a function to simulate a coin flip, returning true if the coin lands on "heads" and false if it lands on "tails." Next, prompt the user for how many trials will be needed for the experiment. Finally, display how many "heads" and "tails" were recorded in the experiment.

Example

User input is underlined.

```
How many coin flips for this experiment: 100
There were 49 heads.
There were 51 tails.
```

Assignment

Please:

- Start from the standard template we use for homework assignments:

```
/home/cs124/tests/templateTest2.cpp
```

- Make sure your professor's name is in the program header.
- Run test bed with

```
testBed cs124/practice21 test2.cpp
```

- Run style checker

Note that the following code might come in handy:

```
#include <stdlib.h> // needed for the rand(), srand()
#include <ctime>    // needed for the time function
int main(int argc, char **argv)
{
    // this code is necessary to set up the random number generator. If
    // your program uses a random number generator, you will need this
    // code. Otherwise, you can safely delete it. Note: this must go in main()
    srand(argc == 1 ? time(NULL) : (int)argv[1][1]);

    // this code will actually generate a random number between 0 and 999
    cout << rand() % 1000 << endl;
}
```

Continued on the next page

Grading for Test2

Sample grading criteria:

	Exceptional 100%	Good 90%	Acceptable 70%	Developing 50%	Missing 0%
Expressions 10%	The expression for the equation is elegant and easy to verify	The expression correctly computes the equation	One bug exists	Two or more bugs exist	The expression is missing
Modularization 20%	Functional cohesion and loose coupling is used throughout	Zero syntax errors in the use of functions, but room exists for improvements in modularization	Data incorrectly passed between functions	At least one bug in the way a function is defined or called	All the code exists in one function
Loop 40%	The loop is both elegant and efficient	The loop is syntactically correct <u>and</u> used correctly	The loop is syntactically correct <u>or</u> is used correctly	Elements of the solution are present	No attempt was made to use a loop
Output 20%	Zero test bed errors	Looks the same on screen, but minor test bed errors	One major test bed error	The program compiles and elements of the solution exist	Program output does not resemble the problem or fails to compile
Style 10%	Well commented, meaningful variable names, effective use of blank lines	Zero style checker errors	One or two minor style checker errors	Code is readable, but serious style infractions	No evidence of the principles of elements of style in the program

Solution available at:

```
/home/cs124/tests/practice21.cpp
```

Unit 2 Project : Calendar Program

Create a calendar for any month of any year from 1753 forward. This is similar to the `cal` utility on the Linux system. Prompt the user for the numeric month and year to be displayed as shown in the example below. Your calculations to determine the first day of the month shall start with January 1, 1753 as a Monday. The challenge here is to take into account [leap years](#).

This project will be done in three phases:

- Project 05 : Design the calendar program
- Project 06 : Compute the offset for a given month and year
- Project 07 : Display the calendar table for a given month and year

Interface Design

There are three parts of the interface design: the output (that which is displayed on the screen during normal operation), the input (that which the user provides), and the errors (when the user enters different data than the program expects).

Output

The following is a sample run of the program. The input is underlined.

```
Enter a month number: 1
Enter year: 1753

January, 1753
Su Mo Tu We Th Fr Sa
   1  2  3  4  5  6
  7  8  9 10 11 12 13
14 15 16 17 18 19 20
21 22 23 24 25 26 27
28 29 30 31
```

The table is formatted in the following way:

January, 1753						
Su	Mo	Tu	We	Th	Fr	Sa
	1	2	3	4	5	6
7	8	9	10	11	12	13
14	15	16	17	18	19	20
21	22	23	24	25	26	27
28	29	30	31			

Input

First the user will be prompted for the month number:

```
Enter a month number: 1
```

Next the user will be prompted for the year:

```
Enter year: 1990
```

Errors

When the program prompts the user for a month, only the values 1 through 12 are accepted. Any other input will yield a re-prompt:

```
Enter a month number: 13  
Month must be between 1 and 12.  
Enter a month number: 0  
Month must be between 1 and 12.  
Enter a month number: 1
```

The same is true with years; the user input must be greater than 1752:

```
Enter year: 90  
Year must be 1753 or later.  
Enter year: 1990
```

Project 05

Write a structure chart for the calendar program. Make sure that all the functions exhibit the highest level of cohesion and the lowest level of coupling.

On campus students are required to attach [this rubric](#) to your design document. Please self-grade.

Project 06

The second part of the Calendar Program project (the first part being the structure part due earlier) is to write the pseudocode for two functions: `computeOffset()` and `displayTable()`.

1. Write the pseudocode for the function `computeOffset`. This function will determine the day of the week of the first day of the month by counting how many days have passed since the 1st of January, 1753 (which is a Monday and `offset == 0`). That number (`numDays`) divided by 7 will tell us how many weeks have passed. The remainder will tell us the offset from Monday. For example, if the month begins on a Thursday, then `offset == 3`. The prototype for the function is:

```
int computeOffset(int month, int year);
```

Please do not plagiarize this from the internet; you must use a loop to solve the problem. The output for this function is the following:

Day	offset
Sunday	6
Monday	0
Tuesday	1
Wednesday	2
Thursday	3
Friday	4
Saturday	5

2. Write the pseudocode for the function `displayTable`. This function will take the number of days in a month (`numDays`) and the offset (`offset`) as parameters and will display the calendar table. For example, consider `numDays == 30` and `offset == 3`. The output would be:

```
Su  Mo  Tu  We  Th  Fr  Sa
    1   2   3
 4   5   6   7   8   9 10
11  12  13  14  15  16 17
18  19  20  21  22  23 24
25  26  27  28  29  30
```

There are two problems you must solve: how to put the spaces before the first day of the month, and how to put the newline character at the end of the week. The prototype of the function is:

```
void displayTable(int offset, int numDays);
```

Project 07

The final part of the Calendar Program project is to write the code necessary to make the calendar appear on the screen:

```
Enter a month number: 1
Enter year: 1753

January, 1753
Su  Mo  Tu  We  Th  Fr  Sa
    1   2   3   4   5   6
 7   8   9  10  11  12  13
14  15  16  17  18  19  20
21  22  23  24  25  26  27
28  29  30  31
```

A few hints:

- Check the case where `offset == 6` when the month begins with Sunday. July 2001 is an example of such a month. You do not want to have a blank line between the column headers (the days of the week) and the day numbers.
- Check the case where the last day of the month is on a Saturday. March 2001 is an example of such a month. You do not want a blank row at the bottom of the calendar.
- Use the `cal` program built into the Linux system when you test the program by hand.

An executable version of the project is available at:

```
/home/cs124/projects/prj07.out
```

Please do the following:

1. Start with the code written in Project 06.
2. Fix any bugs that escaped your notice the first time through.
3. Verify your solution with `testBed`:

```
testBed cs124/project07 project07.cpp
```

4. Submit your code with "Project 07, Calendar" in the program header.